Reproducing GlareShell: Graph Learning-based PHP Webshell Detection, Revisited under Leakage Control

Dang Minh Tu, Vo Quang Vu, Le Tri Duc
University of Information Technology, Ho Chi Minh City, Vietnam
Vietnam National University, Ho Chi Minh City, Vietnam
{24520033,24522045,24520009}@gm.uit.edu.vn

***Abstract*—GlareShell (Feng et al., Computer Networks 2024) detects PHP webshells by turning each script into an interprocedural control-flow graph (ICFG) of statements, embedding the statements, adding a risk level per statement and classifying the graph with a GNN; it reports F1 = 0.9887. Neither code nor data were released. We re-implement the pipeline on the same static-analysis tool (with four fixes needed for modern PHP), rebuild the corpus from the paper's sources with provenance and near-duplicate clusters, and run every experiment with five seeds and paired significance tests. On the paper's protocol we reproduce the headline result (F1 0.98) but not the finer claims: GAT is not better than GCN or GIN, and normalisation and risk weights do not significantly help. Beyond the paper, 77% of test webshells in a random split have a near-duplicate in training, a TF-IDF + logistic-regression baseline beats every GNN, and on four unseen benign projects the false-positive rate quadruples; at a 0.1% alarm budget GAT finds only 6% of unseen webshells (text baseline: 64%), and 10 of 12 hand-written legitimate scripts are flagged. Adding five small benign applications to training cuts the GAT false-positive rate from 2.5% to 0.9% without a significant loss of recall. High scores on this benchmark mostly measure the difference between framework code and webshell collections, not malicious behaviour.**

***Index Terms*—webshell detection, graph neural networks, PHP, reproducibility, data leakage, dataset quality**

I. Introduction

A *webshell* is a server-side script planted in the web root after an initial compromise (unrestricted upload, LFI/RFI, SQL injection, stolen credentials) that gives a remote party control of the server through ordinary HTTP requests (MITRE ATT&CK T1505.003 [1]). Families range from one-line backdoors (`<?php @eval($_POST['x']);?>`) and command shells over large management kits (c99, r57, WSO, b374k) to obfuscated loaders (`eval(gzinflate(base64_decode('...')))`, `$f='as'.'sert'; $f($_REQUEST[k]);`) and plain uploaders. GlareShell [2] targets the Industrial-Internet setting, where the web server is the management plane of control systems, and detects webshells *statically*: a file-system monitor inspects every new or modified `.php` file before it runs. The defender sees only the source file; a false positive blocks a legitimate deployment, a false negative leaves a backdoor.

a) Threat model: Attacker: remote, already able to write a file into the web root, may obfuscate but does not adapt to the specific model. *Defender*: sees only the PHP source at creation/modification time – no execution traces, HTTP logs or file-system context – and must decide per file before it runs. *Out of scope*: memory-resident and non-PHP shells, adaptive attacks, legitimately deployed admin tools. Figure 1 relates the attack chain, the observation surface and the sources of confusion we analyse; Table I lists exactly what GlareShell observes per file. The feature view predicts where it should fail: benign frameworks are full of *obfuscated*-level calls (`call_user_func_array` for dispatch, `usort` with closures); webshells that call dangerous functions only dynamically (`$f(...)`) or hide the payload in an encoded string get no risk signal.

We reproduce GlareShell from its description and ask whether its performance measures what it claims to measure:
RQ1 Can the reported performance (Tables 2 and 4 of [2]) be reproduced from the paper's sources?
RQ2 Do normalisation, risk weights, the embedding and the GNN contribute as claimed?
RQ3 How much of the score is explained by near-duplicate leakage and by the source confound?
RQ4 How does the detector behave on benign projects and webshells outside the training sources?
Contributions: (1) a working re-implementation including four extractor fixes; (2) a rebuilt, documented corpus (GlareShell-R) with hashes, provenance, near-duplicate clusters and leakage-controlled splits; (3) a 5-seed replication with paired tests; (4) leakage, baseline, alarm-budget and out-of-source analyses; (5) an improvement experiment. An extended version with all tables (corpus funnel, extraction statistics, sweeps, ablation, per-project FPR, error taxonomy, hyper-parameters) is in the repository (`report/extended.pdf`).

II. GlareShell and our re-implementation

a) Original pipeline: (1) *ICFG extractor*: LChecker [3] (on PHP-Parser [4]) builds one CFG per function; the paper's Algorithm 1 links them through call and return edges into an ICFG whose nodes are PHP statements, and "meaningless" statements are pruned. (2) *Node attributes*: statements are normalised (local variables → `local`, user functions → `user-define`, numbers → `im`) and embedded with one of six models (Word2Vec CBOW/Skip-gram [5], FastText [6], GloVe [7], Doc2Vec DM/DBOW [8]); a one-hot risk level is appended: *critical* (`system`, `exec`, `shell_exec`, `passthru`, `popen`, backticks, ...), *obfuscated* (`eval`,

Course project for NT230 (Malware Analysis); instructor: Phan The Duy.

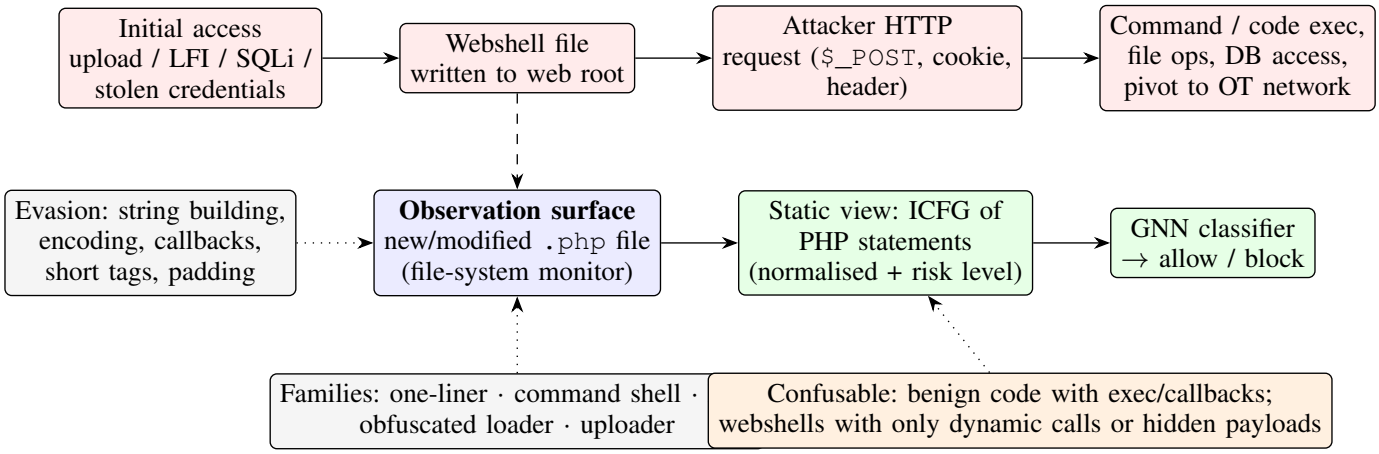


Fig. 1. Concept map: attack chain (red), what the detector observes (blue), the detector (green) and the sources of confusion we analyse (orange).

TABLE I
FEATURE VIEW OF GLARESHELL (PER NODE OF THE ICFG UNLESS STATED).

Feature	Content	Malicious signal it can carry
Statement text	One PHP statement / branch condition, pretty-printed	API used, request input, payload strings
Normalisation	local variables → `$local`, user functions/methods → `user_define`, numbers → `im`	removes names/passwords, keeps built-ins and strings
Embedding	100-d vector of the normalised statement (W2V / FastText / GloVe / D2V)	co-occurrence of risky statements
Risk level	one-hot: 2 = *critical* (`system`, `exec`, `shell_exec`, `passthru`, `popen`, `proc_open`, `pcntl_exec`, backticks); 1 = *obfuscated* (`eval`, `assert`, `preg_replace`, `create_function`, `call_user_func*`, array/sort callbacks, …); 0 = normal	explicit dangerous capability
Edges	intra-procedural control flow + call/return edges between functions (ICFG)	how input reaches the sink
Graph	mean of final node states (Eq. 6 of [2])	whole-file behaviour

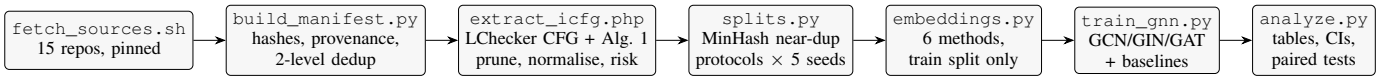


Fig. 2. Reproduction pipeline (one script per box in `src/`). Samples are only parsed; nothing is executed.

`assert`, `create_function`, `call_user_func*`, callbacks, …) or normal. (3) *Classifier*: GCN [9], GAT [10] or GIN [11], mean pooling and an MLP. Reported: FastText best embedding (F1 0.945), GAT best GNN (F1 0.989) on 3,104 webshells and 10,375 benign files split randomly 8:1:1.

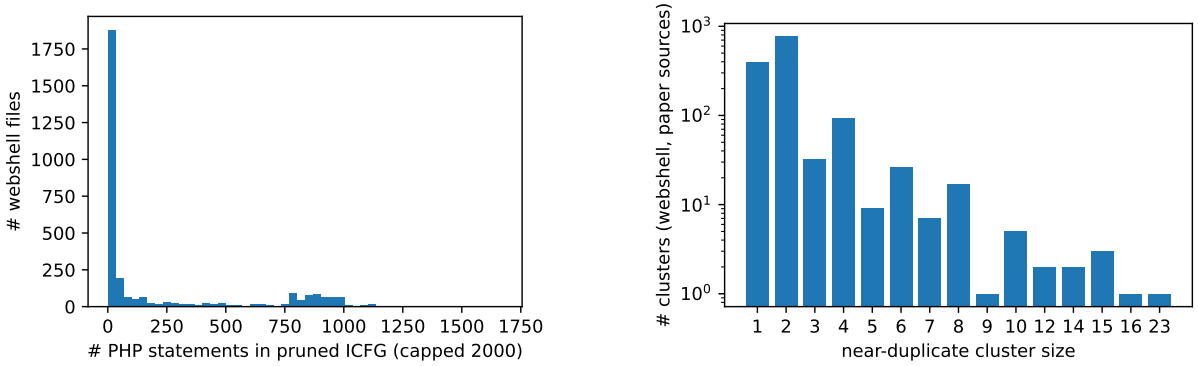
b) Our implementation (Figure 2): LChecker's own analysis is a taint checker that outputs no graphs, but its CFG builder is reusable; we call it unchanged and implement Algorithm 1, pruning, AST-level normalisation and risk levels on top (`src/php/extract_icfg.php`). Four problems not mentioned by the paper had to be fixed: (i) LChecker silently skips `namespace`, `try/catch` and `declare` blocks, so vanilla LChecker yields *empty graphs for most modern framework files* – a perfect label leak; we flatten these blocks. (ii) `for` conditions crash the pretty-printer (854 files). (iii) PHP 8 disables short open tags, so 208 webshells (and no benign file) parse as HTML; we parse with `short_open_tag=On`. (iv) Two files segfault PHP and are kept as empty graphs. ICFGs are capped at 2,000 statements (15% of webshells, 0.4% of benign files). Embeddings use Gensim defaults (dimension 100, window 5) and are **trained only on the training files of each split and seed**, so no test statement leaks into them. GNNs use PyTorch Geometric with the paper's hyper-parameters (Table 2 setting: 2-layer GCN, hidden 10; Table 3 optimum: 3 layers, batch 256, 120 epochs); model selection uses validation F1.

III. DATASET: GLARESHELL-R

a) Sources: We cloned every repository of the paper's Table 1 at pinned revisions. `ysrc/webshell-sample` has been deleted; its files survive only inside the Cyc1e183 aggregate [12] (2,917 webshells from 17 collections, named by MD5, no origin), which dominates the webshell side. Benign files come from WordPress, Drupal, Laravel, PHPMailer and doorGets. After exact-hash deduplication we obtain **3,097**

TABLE II
ICFG EXTRACTION OUTCOME (AFTER THE FOUR EXTRACTOR FIXES OF SECTION II).

Label	Subset	Files	Crash	Parse err.	Empty ICFG	Truncated	Median nodes	P90 nodes	ms/file
benign	heldout	9101	0	1	123	7	18	103	6.9
benign	paper	12930	0	1	590	49	14	99	7.9
webshell	heldout	46	0	7	3	4	25	755	26.6
webshell	paper	3097	4	139	53	477	11	875	36.6



(a) Statements per webshell ICFG (cf. paper Fig. 5).

(b) Near-duplicate cluster sizes, paper webshells.

Fig. 3. Corpus structure.

webshells (paper: 3,104, so the paper used exact deduplication) and subsample 10,375 benign files as in the paper. For generalisation we add four *held-out* benign projects (Laravel framework, phpMyAdmin, Joomla, Matomo) and two held-out webshell collections. Every file has a SHA-256, source, revision and path in the manifest; a data card, schema and label guide are in `docs/`.

b) Extraction is label-dependent: Table II shows that webshells are older, less well-formed and much larger: 4.5% contain syntax errors, 15.4% exceed the 2,000-statement cap (benign: 0.4%), and their statement counts span four orders of magnitude (Figure 3a). These are signals a classifier can exploit without understanding behaviour.

c) Near-duplicates: 1,265 of the 2,917 Cyc1e183 files (43%) are identical once whitespace is removed. Clustering files by the set of their normalised statements (MinHash-LSH [13], Jaccard ≥ 0.8) collapses the 3,097 webshells into 1,368 clusters (largest: 55 members, Figure 3b) (benign: 12,930 $\rightarrow$ 11,828). In the paper's random split **77.0% $\pm$ 4.4% of test webshells have a near-duplicate in training** (benign: 9%), so its test set mostly measures recognition of known families [14], [15].

d) Evaluation protocols: (5 seeds each)

`paper_random` the paper's protocol: exact dedup, stratified random 8:1:1.

`clean_group` leakage-controlled: whitespace dedup; whole near-duplicate clusters go to train, validation or test.

`cross_source` generalisation: trained as `clean_group`; tested on 9,091 files of the four unseen benign projects and the 18 webshells that share no cluster with the paper corpus – primarily a *false-positive* stress test.

`loo` leave-one-collection-out: all webshells first published in one collection (tennc 222, bartblaze 99, xl7dev 77) and every cluster they touch are held out.

A temporal split is impossible: 93% of webshells share one commit date (the Cyc1e183 upload, 2020-11-13).

IV. EXPERIMENTAL SETUP

Hardware: RTX 3050 6 GB, 7.6 GB RAM; PyTorch 2.4.1, PyG 2.6.1, Gensim 4.3.3, PHP 8.3. Positive class = webshell; we report the paper's precision, recall, TNR and F1 plus FPR, the quantity that decides deployability. Every configuration runs with 5 seeds (mean $\pm$ 95% Student-t half-width); key comparisons use a paired t-test on per-seed F1 and an exact McNemar test on per-file decisions. **Baselines** on the same splits: TF-IDF of lexical PHP tokens (1–2-grams) with logistic regression or XGBoost; rules built from GlareShell's own risk lists; and *graph statistics* + XGBoost – twelve numbers per ICFG (size, density, number of risky statements, parse error, ...) with no code semantics, a probe for corpus artefacts.

V. RESULTS

A. Reproducing the paper (RQ1, RQ2)

Table III compares our F1 with the paper. The headline is reproduced: all three GNNs reach F1 $\approx$ 0.98 (paper 0.968–0.989), and FastText is the best embedding by a significant margin (paired $p = 0.007$, McNemar significant in 5/5 seeds). The finer claims are not: (i) the five other embeddings lie within 0.036 F1 with overlapping intervals and a different order; (ii) GAT is *not* the best GNN – GAT vs. GCN $p = 0.79$, vs. GIN $p = 0.48$; (iii) F1 is flat over embedding dimensions

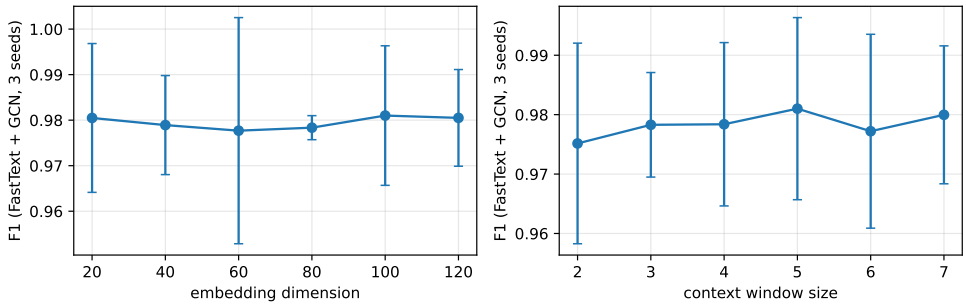


Fig. 4. Embedding dimension (left) and FastText context window (right) vs. F1, FastText + GCN on `paper_random`, 3 seeds (cf. paper Fig. 6 and 7). Error bars: 95% CI.

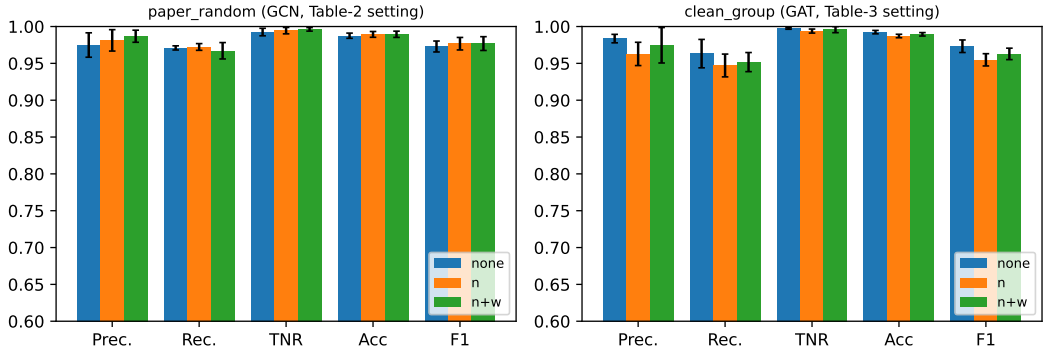


Fig. 5. Ablation of normalisation and risk weights on both protocols (cf. paper Fig. 8).

TABLE III
F1 ON PAPER_RANDOM (5 SEEDS) VS. THE PAPER. TOP: TABLE 2 (EMBEDDINGS, 2-LAYER GCN); BOTTOM: TABLE 4 (FASTTEXT).

Setting	Ours	Paper	Δ
GloVe	0.928 ± 0.013	0.914	+0.015
Word2Vec CBOW	0.899 ± 0.025	0.937	−0.038
Word2Vec Skip-gram	0.915 ± 0.012	0.936	−0.021
Doc2Vec DM	0.912 ± 0.020	0.931	−0.019
Doc2Vec DBOW	0.935 ± 0.018	0.929	+0.006
FastText	**0.977 ± 0.009**	**0.945**	+0.032
GCN	0.981 ± 0.008	0.968	+0.013
GIN	0.983 ± 0.002	0.977	+0.007
GAT	0.980 ± 0.012	**0.989**	−0.009

20–120 (0.978–0.981) and windows 2–7, with no rise-then-fall (Figure 4); (iv) normalisation and risk weights do not significantly help ($p = 0.10$ on `paper_random`), and on `clean_group` raw statements without either are nominally best ($p = 0.07$, Figure 5); normalisation removes names and literals that help to recognise near-duplicates in a random split. Table IV lists all paired tests.

B. Leakage, baselines and generalisation (RQ3, RQ4)

a) Leakage and baselines (Table V): Removing near-duplicate leakage (`clean_group`) lowers GNN F1 by about two points (GAT 0.980 → 0.963). A bag of lexical tokens with logistic regression beats every GNN on every protocol (F1 0.997 / 0.992; paired $p \leq 0.019$), at seconds of CPU time and without an ICFG. Twelve graph statistics without any code semantics reach F1 0.917 on `paper_random`: most of the separation is available from the *shape* of the ICFG. Rules from GlareShell's own risk lists are weak (F1 0.36–0.64): many benign framework files call the same functions.

b) Unseen benign projects and alarm budgets: On `cross_source` the GNN false-positive rate quadruples, from 0.4% to 1.7% (Joomla worst at 3%), while TF-IDF + LR stays at 0.35%. The threshold-free view is worse (Figure 6): with a 0.1% alarm budget, the realistic budget of a file-upload monitor, GAT recovers only **6%** of the unseen webshells (average precision 0.11), TF-IDF + LR **64%** (AP 0.61). The GNN's false positives on unseen projects are *confident*: they rank above most webshells, so no threshold separates them.

c) Unseen webshells: Recall on the 18 novel webshells drops from 0.97 to 0.64 (GAT) and 0.54 (GCN). Holding out whole collections (`loo`) is milder – GNN recall 0.92–0.95 on 77–222 unseen webshells – but TF-IDF again does better (0.97–1.00) and even graph statistics recover 69–87%: public collections copy from each other, so a held-out collection is not a held-out family.

d) Small standalone benign scripts: All benign training files come from large frameworks. We wrote 12 ordinary standalone scripts of the kind found on small company servers (contact form, image upload with MIME check, PDO login, CSV export, JSON API, admin ping tool with

TABLE IV
PAIRED SIGNIFICANCE TESTS (5 SEEDS, SAME TEST FILES). "SIGNIFICANT": PAIRED-t $p < 0.05$.

Protocol	Comparison (A vs B)	ΔF1 (A−B)	paired-t p	McNemar $p < .05$	Significant
paper_random	GAT vs GCN (Table 4)	-0.0009±0.0087	0.790	0/5	no
paper_random	GAT vs GIN (Table 4)	-0.0034±0.0123	0.479	1/5	no
paper_random	FastText vs D2V-w (Table 2)	+0.0419±0.0226	0.007	5/5	yes
paper_random	n+w vs none (GCN, Fig. 8)	+0.0040±0.0052	0.100	1/5	no
clean_group	n+w vs none (GAT, Fig. 8)	-0.0104±0.0116	0.068	1/5	no
paper_random	GAT vs TF-IDF + LR	-0.0174±0.0126	0.019	3/5	yes
clean_group	GAT vs TF-IDF + LR	-0.0291±0.0100	0.001	4/5	yes
cross_source	GAT vs TF-IDF + LR	-0.2994±0.0390	0.000	5/5	yes

TABLE V
GNNS AND BASELINES (5 SEEDS). F1 ON THE PAPER'S RANDOM SPLIT (PAPER_RANDOM) AND THE CLEAN SPLIT (CLEAN_GROUP); FPR ON 9,091 FILES OF FOUR UNSEEN BENIGN PROJECTS AND RECALL ON 18 UNSEEN WEBSHELLS (CROSS_SOURCE).

Model	F1 random	F1 clean	FPR unseen	Recall unseen
GlareShell GCN	0.981±.008	0.959±.011	1.84%	0.54±.06
GlareShell GAT	0.980±.012	0.963±.008	1.69%	0.64±.13
TF-IDF + LR	**0.997**±.002	**0.992**±.005	**0.35%**	0.77±.03
TF-IDF + XGBoost	**0.998**±.002	0.990±.008	1.34%	**0.92**±.04
Graph stats + XGB	0.917±.008	0.825±.043	2.88%	0.42±.08
Rule: any risky call	0.637±.010	0.551±.077	9.98%	0.22

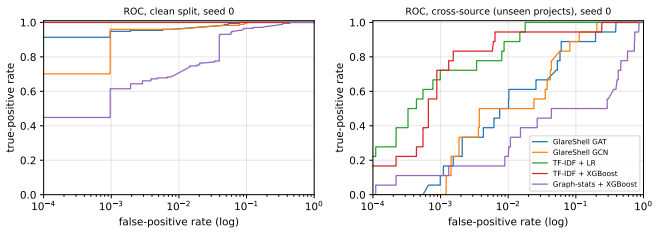


Fig. 6. ROC curves (log FPR axis), seed 0. Left: `clean_group`; right: `cross_source` (unseen benign projects, 18 unseen webshells).

`escapeshellarg`, a three-line greeting, . . .). The final demo model (FastText + GAT, F1 0.974 and FPR 0.1% on its own test set) flags **10 of the 12** as webshells, most with $p > 0.98$; TF-IDF + LR flags 6 (Table VI). Both detectors have learned the population difference – small, procedural, reads `$_GET/$_POST` – rather than malicious behaviour.

e) Error analysis: Missed webshells are mostly files the pipeline cannot see into: on `paper_random` 75% have an empty ICFG (PHP 4-era or malformed files, encoded droppers whose payload is a string) and 86% contain no call from the risk lists. About half of the `cross_source` misses we inspected are *label noise*: helper files shipped inside a webshell kit (`phpinfo.php`, a database class). False positives on unseen projects (769 over 5 seeds) are 62% Joomla, and 47% carry no risk tag at all: layout templates mixing HTML and short PHP blocks, and small event or exception class stubs. False positives are driven by *distance from the benign training projects*, not by risky code.

C. Improvement: training on small benign applications

The analysis suggests a fix: show the model benign code that resembles the deployment. We added five small procedural PHP applications to training (YOURLS, FluxBB, DokuWiki, Cacti, Adminer; 2,511 files) and kept four others for testing only (OPNsense core, osTicket, MyBB, Tiny File Manager; 2,709 files), removing any file that also occurs in the main corpus. Base and augmented models are evaluated on the *same* 13,041 files (`clean_group` and `cross_source` tests, held-out apps, probes). Pooled over all benign test files, the GAT false-positive rate falls from 2.5% to 0.9% (paired $p = 0.006$) with no significant change in recall ($p = 0.25$); TF-IDF + LR falls from 1.7% to 0.5% but loses 2.5 recall points (Table VII). The gain carries over to the large unseen projects (1.45% → 0.63%). It is not a cure: GAT still flags 8 of the 12 probe scripts, and Tiny File Manager, a legitimate single-file file manager, stays flagged.

VI. DISCUSSION

Table VIII summarises the verdicts.

a) Threats to validity: Internal: the ICFG construction, risk lists and statement format are our reading of an unreleased implementation; hyper-parameters the paper leaves open were fixed before looking at test results. *Construct*: labels are source-level – "webshell" means "from a public collection" – and the two populations also differ in age, size and parseability, which a classifier can exploit; collections contain non-malicious helpers. *External*: five benign training projects (ten with the improvement) and only 18 truly novel webshells; no adaptive attacker. *Conclusion*: five seeds give limited power, so a non-significant difference means "not shown", not "equal"; only the alarm-budget operating points use test-set thresholds, which is optimistic for all models alike.

b) Safety, ethics and reproducibility: Samples are only parsed, never executed or served; `samples/` and derived code fragments are git-ignored, the Docker image runs with `-network none` and read-only samples, and graders receive a password-protected archive (`infected`) with checksums. The repository contains pinned fetch scripts, the hash manifest, all splits, every run's configuration and per-file predictions, a Dockerfile and `reproduce.sh`; `analyze.py` regenerates every table and figure. The 125 GNN runs took about 5.3 GPU-hours on a laptop GPU.

TABLE VI
WEBSHELL PROBABILITY FOR 12 HAND-WRITTEN BENIGN STANDALONE SCRIPTS (`tests/probe_benign/`); MODELS TRAINED ON `clean_group`, SEED 0. BOLD: FLAGGED ($p \geq 0.5$).

Script	GAT	TF-IDF+LR	What it does
hello.php	**1.000**	**0.924**	echo greeting with `htmlspecialchars`
contact_form.php	**0.998**	**0.760**	validated form + `mail()`
download.php	**0.999**	0.373	whitelisted file download
export_csv.php	**0.996**	**0.569**	PDO query → CSV
guestbook.php	**0.998**	**0.916**	append-only text file
image_upload.php	**0.982**	0.252	MIME/size-checked upload
login.php	**0.930**	**0.651**	PDO + `password_verify`
ping_tool.php	**0.992**	**0.628**	`shell_exec` with validated IP + `escapeshellarg`
router.php	**0.994**	0.146	static route table + `include`
template_render.php	**0.999**	0.201	`extract` + `include` template
cleanup_cron.php	0.376	0.490	delete old cache files
json_api.php	0.204	0.377	JSON to-do list API
Flagged	**10/12**	**6/12**	

TABLE VII
BASE (`clean_group` TRAINING) VS. AUG (+5 SMALL BENIGN APPS); SAME 13,041 TEST FILES, MEAN OVER 5 SEEDS.

	GAT		TF-IDF + LR	
Test group	base	aug	base	aug
All benign files: FPR	2.52%	**0.92%**	1.71%	**0.55%**
All webshells: recall	0.926	0.908	0.971	0.946
Unseen large projects: FPR	1.45%	0.63%	0.34%	0.15%
osTicket: FPR	7.35%	1.32%	12.26%	3.67%
OPNsense core: FPR	5.67%	2.70%	0.24%	0.14%
MyBB: FPR	3.81%	1.09%	1.86%	0.19%
12 probe scripts flagged	11.2	8.4	6.2	3.0
Tiny File Manager p	0.95	0.63	0.71	0.53

VII. CONCLUSION

GlareShell's headline F1 of about 0.98 is reproducible without the authors' code or data; its finer conclusions are not, once seeds and paired tests are used. More importantly, the benchmark mostly measures the difference between five large frameworks and public webshell collections: 77% of test webshells leak via near-duplicates, a lexical baseline beats the GNNs, and the models flag most small legitimate scripts and lose almost all recall at a realistic alarm budget on unseen projects. Training on benign code that resembles the deployment cuts false positives by a factor of almost three. A practical detector needs such benign data at scale, cluster- and source-disjoint evaluation by default, relabelled kit helpers, and FPR on unseen projects reported next to F1.

REFERENCES

[1] MITRE ATT&CK, "Server software component: Web shell (T1505.003)," https://attack.mitre.org/techniques/T1505/003/, 2024.

[2] P. Feng, D. Wei, Q. Li, Q. Wang, Y. Hu, N. Xi, and J. Ma, "GlareShell: Graph learning-based PHP webshell detection for web server of industrial internet," *Computer Networks*, vol. 245, p. 110406, 2024.

[3] P. Li and W. Meng, "LChecker: Detecting loose comparison bugs in PHP," in *Proceedings of The Web Conference (WWW)*, 2021.

[4] N. Popov, "PHP-Parser: A PHP parser written in PHP," https://github.com/nikic/PHP-Parser, v4.19.4, 2024.

[5] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *NeurIPS*, 2013.

[6] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *TACL*, vol. 5, pp. 135–146, 2017.

[7] J. Pennington, R. Socher, and C. Manning, "GloVe: Global vectors for word representation," in *EMNLP*, 2014.

[8] Q. Le and T. Mikolov, "Distributed representations of sentences and documents," in *ICML*, 2014.

[9] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *ICLR*, 2017.

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *ICLR*, 2018.

[11] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *ICLR*, 2019.

[12] Cyc1e183, "PHP-Webshell-Dataset," https://github.com/Cyc1e183/PHP-Webshell-Dataset, commit 59ff14f, 2021.

[13] A. Z. Broder, "On the resemblance and containment of documents," in *Compression and Complexity of Sequences*, 1997.

[14] M. Allamanis, "The adverse effects of code duplication in machine learning models of code," in *Onward! (SPLASH)*, 2019.

[15] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *USENIX Security Symposium*, 2022.

TABLE VIII

VERDICT PER CLAIM OF THE PAPER AND PER ADDITION OF THIS WORK. **GREEN**: REPRODUCED; **RED**: NOT REPRODUCED; **BLUE**: IMPROVED OR EXTENDED.

Claim / item	Our evidence	Verdict
ICFG + embedding + GNN reaches F1 $\approx$ 0.95–0.99	F1 0.980–0.983 on the paper's protocol, 5 seeds	**Reproduced**
FastText is the best embedding	0.977, best of six, $p = 0.007$	**Reproduced**
GAT is the best GNN	GCN 0.981, GIN 0.983, GAT 0.980; $p = 0.79$ / 0.48	**Not reproduced**
Dimension / window shape (Fig. 6–7)	flat; window 5 nominal maximum only	**Not reproduced**
Normalisation + risk weights help (Fig. 8)	$p = 0.10$ (`paper_random`), reversed on `clean_group` ($p = 0.07$)	**Not reproduced**
GNN beats text approaches	TF-IDF + LR better on every protocol, significant	**Not reproduced**
Cheap enough for online monitoring	11 ms extraction + ms scoring per file	**Reproduced**
Extractor for PHP 8 code	four LChecker fixes, documented	**Improved**
Corpus, leakage control, new protocols	GlareShell-R; 77% leakage; `clean_group`, `cross_source`, `loo`	**Extended**
Deployment view	FPR $\times 4$ on unseen projects; 6% recall at 0.1% FPR; 10/12 probes flagged	**Extended**
Training on small benign applications	FPR 2.5% $\rightarrow$ 0.9% ($p = 0.006$), recall unchanged	**Improved**